Contents

YipitData AI Engineer Take-Home Exercise **1**
- What you're doing . . . 1
- The customer's question . . . 1
- What you send back . . . 1
- How long this should take . . . 2
- Tools you can use . . . 2
- The data . . . 2
  - Free API key 1: FRED (Federal Reserve Economic Data) . . . 2
  - No key needed: yfinance (Yahoo Finance) . . . 3
  - No key needed: SEC EDGAR . . . 4
  - Optional: Alpha Vantage or Financial Modeling Prep . . . 5
- How to think about the work . . . 5
  - A simple, partly-wrong starting approach . . . 6
  - Why the simple approach is wrong . . . 6
  - What "good" looks like, roughly . . . 7
- What happens after you submit . . . 7
- What we don't care about . . . 7
- How to submit . . . 7
- Questions . . . 8

YipitData AI Engineer Take-Home Exercise

What you're doing

We hand you a real customer question and a small set of files. You spend 4 to 6 hours on it. You send back three things: a notebook, a one-page memo, and a prompt log. We read what you sent. Then we talk it through with you on a Zoom call.

The question is closer to what we actually do here than it is to a textbook problem. There is no single right answer.

The customer's question

Read this brief carefully. It is the actual question.

> "We track the monthly retail-sales series from FRED for our subsector. We are thinking about using it as a leading indicator of quarterly revenue for Walmart. Does it predict Walmart's revenue better than a naive baseline? If yes, by how much, and what should we worry about? If no, what evidence would change our minds?"

A "leading indicator" is a signal that moves before the thing you care about. A "naive baseline" is the simplest possible guess. We'll show you a few examples of each below.

What you send back

Three files, packed into one zip:

1. **A Python notebook** called `analysis.ipynb`. Your full analysis lives here. The code should run end to end without errors. Add comments where they help a human reader follow you.
2. **A one-page memo** called `memo.md` or `memo.pdf`. Frame the question, give the answer, list the things that worry you. Imagine the reader is a portfolio manager who took stats in college but has not done much of it since.
3. **A prompt log** called `prompts.md`. Paste/exports the prompts you sent to your LLM assistant during the work. Add a short note (under 200 words) about what the assistant got right, where you had to push back, and how you checked its output. Also acceptable to bundle your exported prompts in a folder or compressed file, and write up only your note in `prompts.md`.

How long this should take

Plan for 4 to 6 hours. Hard cap at 6.

If you go over, tell us. We read your work for judgment, not for stamina.

Tools you can use

Python is required. Beyond that, work the way you actually work.

You can use any LLM coding assistant: Claude Code, Cursor, GitHub Copilot, ChatGPT, whichever one you reach for. We expect that you will. We are testing your judgment about LLMs, not your willingness to do without them. The prompt log is the artifact that shows us how you used the tool.

The data

Pull and rename two CSV files using the directions below.

- `retail_sales_fred.csv`: monthly U.S. retail-sales index from FRED (series RSXFS), from 2010 through last month
- `walmart_revenue.csv`: quarterly revenue for Walmart (ticker WMT), pulled from SEC filings, from 2010 through the most recent reported quarter

You can use these files as they are. That is the easy path.

If you would rather pull fresher data, or try a different retailer, the next section walks you through the free APIs you would need. You do not have to do this. The CSVs are enough to do strong work.

Free API key 1: FRED (Federal Reserve Economic Data)

FRED is the public data archive run by the St. Louis Fed. Signing up takes about a minute.

1. Go to https://fred.stlouisfed.org/docs/api/api_key.html
2. Click “Register for an account.”
3. Confirm your email.
4. Click “Request API Key.”
5. Copy the key. It looks like a string of 32 letters and digits.

Once you have the key, you can call FRED from Python like this:

```
"""monthly U.S. retail-sales index (RSXFS), Jan 2010 through last released
 ↪  month."""

from pathlib import Path

import pandas as pd
import requests

FRED_KEY = "paste_your_key_here"  #
 ↪  https://fred.stlouisfed.org/docs/api/api_key.html

url = "https://api.stlouisfed.org/fred/series/observations"
params = {
    "series_id": "RSXFS",
    "api_key": FRED_KEY,
    "file_type": "json",
    "observation_start": "2010-01-01",
}
r = requests.get(url, params=params, timeout=30)
r.raise_for_status()

df = pd.DataFrame(r.json()["observations"])[["date", "value"]]
df["date"] = pd.to_datetime(df["date"])
df["value"] = pd.to_numeric(df["value"], errors="coerce")
df = df.dropna().sort_values("date").reset_index(drop=True)

Path("data").mkdir(exist_ok=True)
df.to_csv("data/retail_sales_fred.csv", index=False)
```

If you prefer a wrapper, `fredapi` or `pandas-datareader` both work fine.

No key needed: yfinance (Yahoo Finance)

The yfinance Python package does not need an API key. It pulls quarterly financials straight from Yahoo.

```
import yfinance as yf

wmt = yf.Ticker("WMT")
quarterly = wmt.quarterly_financials   # rows are line items, columns are
 ↪  quarter-end dates
revenue = quarterly.loc["Total Revenue"]
```

Heads up: yfinance scrapes a public web page. It breaks once or twice a year when Yahoo changes the layout. If a call fails, run `pip install -U yfinance` to get the latest version and try again.

No key needed: SEC EDGAR

EDGAR is the SEC's filings archive. There is no key. EDGAR does require a User-Agent header on every request so the SEC knows who is hitting the server.

```
"""quarterly Walmart revenue (fiscal Q1–Q4), Jan 2010 through most recent
 ↪  reported quarter.

two pieces the simple snippet hides:
  - ASC 606 concept switch: Walmart files revenue under us-gaap:Revenues
 ↪  through
    fiscal 2018 and
 ↪  us-gaap:RevenueFromContractWithCustomerExcludingAssessedTax
    from fiscal 2019 onward. one concept alone yields an eight-year gap.
  - Q4 is not filed quarterly. XBRL provides Q1, Q2, Q3 plus the annual FY
 ↪  total.
    Q4 must be derived as FY − (Q1 + Q2 + Q3) on matching fiscal-year
 ↪  boundaries.
"""

from pathlib import Path

import pandas as pd
import requests

USER_AGENT = "Your Name your.email@example.com"  # SEC requires this header

frames = []
for concept in ("Revenues",
                "RevenueFromContractWithCustomerExcludingAssessedTax"):
    url = ("https://data.sec.gov/api/xbrl/companyconcept/"
           f"CIK0000104169/us-gaap/{concept}.json")
    r = requests.get(url, headers={"User-Agent": USER_AGENT}, timeout=30)
    if r.status_code == 404:
        continue
    r.raise_for_status()
    units = r.json().get("units", {}).get("USD", [])
    if units:
        frames.append(pd.DataFrame(units))
raw = pd.concat(frames, ignore_index=True)

raw["start"] = pd.to_datetime(raw["start"])
raw["end"] = pd.to_datetime(raw["end"])
raw["filed"] = pd.to_datetime(raw["filed"])
raw["days"] = (raw["end"] - raw["start"]).dt.days

# quarterly facts ~ 89–94 days; fiscal-year facts ~ 360–371 days
raw = raw[raw["days"].between(80, 100) | raw["days"].between(355,
 ↪  375)].copy()
```

```
raw["kind"] = raw["days"].apply(lambda d: "Q" if d <= 100 else "FY")
# dedupe amendments: keep the latest filing per (start, end, kind)
raw = raw.sort_values("filed").drop_duplicates(["start", "end", "kind"],
 ↪  keep="last")

q_facts = (raw[raw["kind"] == "Q"][["end", "val"]]
           .rename(columns={"end": "date", "val": "value"}))

# derive Q4 = FY - (Q1 + Q2 + Q3) for each fiscal year
q4_rows = []
for _, fy in raw[raw["kind"] == "FY"].iterrows():
    in_fy = q_facts[(q_facts["date"] > fy["start"]) & (q_facts["date"] <
 ↪  fy["end"])]
    if len(in_fy) == 3:
        q4_rows.append({"date": fy["end"],
                        "value": float(fy["val"]) -
                         ↪  float(in_fy["value"].sum())})

revenue = pd.concat([q_facts, pd.DataFrame(q4_rows)], ignore_index=True)
revenue["value"] = revenue["value"].astype(float)
revenue = (revenue.drop_duplicates("date")
                  .sort_values("date")
                  .query("date >= '2010-01-01'")
                  .reset_index(drop=True))

Path("data").mkdir(exist_ok=True)
revenue.to_csv("data/walmart_revenue.csv", index=False)
```

The CIK above (`0000104169`) is Walmart. You can look up other CIKs at https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany.

Optional: Alpha Vantage or Financial Modeling Prep

You do not need either of these for this exercise. They are useful to know about.

- Alpha Vantage: free key at https://www.alphavantage.co/support/#api-key. Limit: 5 calls per minute, 500 per day on the free tier.
- Financial Modeling Prep (FMP): free key at https://site.financialmodelingprep.com/developer/docs. Limit: 250 calls per day on the free tier.

Both wrap finance data behind a tidy REST API. Both have rough edges around the financials endpoints. Use them only if you already know them.

How to think about the work

Here is one way to start. It is not the only way, and we do not think it is the best way. We give it to you so you have something to react to.

A simple, partly-wrong starting approach

Imagine you wrote the following code as a first pass:

```
import pandas as pd
import statsmodels.api as sm

retail = pd.read_csv("retail_sales_fred.csv", parse_dates=["date"])
revenue = pd.read_csv("walmart_revenue.csv", parse_dates=["date"])

# resample retail sales from monthly to quarterly
retail_q = (retail.set_index("date")
                  .resample("QE")["value"].sum()
                  .reset_index())

# year-over-year growth, both series
retail_q["yoy"] = retail_q["value"].pct_change(4)
revenue["yoy"] = revenue["value"].pct_change(4)

merged = pd.merge(retail_q, revenue, on="date", suffixes=("_retail",
 ↪  "_rev"))

X = sm.add_constant(merged["yoy_retail"])
y = merged["yoy_rev"]
model = sm.OLS(y, X, missing="drop").fit()
print(model.summary())  # report R-squared
```

That gets you a first pass. It is also wrong in at least four ways. See if you can name them before you read the next part.

Why the simple approach is wrong

Four problems, in rough order of how much they matter.

1. **No baseline.** “How well does X predict Y?” is meaningless without a baseline to beat. The right question is: does this signal beat a naive baseline like “next quarter’s revenue equals last year’s same quarter, plus typical growth”? An R-squared of 0.7 sounds great until you find out a seasonal-naive baseline already gets 0.85 on the same data.
2. **In-sample evaluation.** Fitting a regression on all of history and reporting R-squared tells you how well the model fits the past. It says nothing about how well it predicts the future. You need an out-of-sample test: train on data through year X, predict year X+1, repeat.
3. **Look-ahead bias.** Walmart’s quarterly revenue for Q3 is reported in mid-Q4. If you treat that number as known on the last day of Q3, you have leaked the future into your features. Be honest about when each data point was actually available.
4. **No story for “why.”** Even if the signal predicts well, you should be able to say something about why. Is retail sales causing the relationship, or just correlated through a third factor? Is the relationship stable across years, or did it break in 2020 with the pandemic?

What "good" looks like, roughly

A solid submission will likely:

- Pick a real baseline and beat it, or honestly say the signal did not beat one
- Run the test out of sample with a proper time-series split (no shuffled k-fold for time series, please)
- State the caveats out loud, not bury them in footnotes
- Use one or two clean figures, not ten messy ones
- Run end to end without errors when we re-run the notebook

A great submission will also:

- Tell us something we did not already know about when the signal works and when it does not
- Show one or two places where the LLM got something subtly wrong and the candidate caught it
- Land at a clear, falsifiable claim ("The signal beats the seasonal-naive baseline by X percent on out-of-sample MAPE, but only outside recession periods")

What happens after you submit

If your work exceeds our threshold, we set up a 60-minute Zoom. The first half hour, you walk us through what you did. The second half hour, we pick one decision you made and push back on it. We are looking for two things:

1. Can you defend the choices you made with technical reasoning?
2. When we point out something you missed, do you update on it cleanly? Anchoring to a wrong answer or folding without resistance both look bad. Calm reasoning under disagreement looks great.

What we don't care about

- Polishing every figure to magazine quality. Clean is enough.
- Running every model you have ever heard of. Pick one or two and do them well.
- Hiding your use of LLMs. Show us the prompts. We mean it.
- Going over the time cap to make it perfect. Stop and submit.

How to submit

Send a single zip file named `firstname_lastname_yipitdata.zip` to the email address Recruiting gave you. The zip should look like this:

```
firstname_lastname_yipitdata/
├── analysis.ipynb
├── memo.md (or memo.pdf)
├── prompts.md
└── data/
    ├── retail_sales_fred.csv
    └── walmart_revenue.csv
```

If you used Python packages beyond pandas, numpy, matplotlib, statsmodels, and scikit-learn, include a `requirements.txt` so we can reproduce your environment.

Questions

If anything in this brief is unclear, email Recruiting. We would rather answer one question than read ten guesses about what we meant.

Good luck. Have fun with it.